Technical Design Document

Loan Navigator Agent Suite

Project	Loan Navigator Agent Suite
Client	BlueLoans4all
LLM / Embeddings	Gemini 2.5 Flash / gemini-embedding-001
Orchestration	LangGraph StateGraph + FastAPI + Streamlit
Deployment	GCP Cloud Run + Artifact Registry + Cloud Build
Classification	Confidential
Team Members (Group 1)	1. Aatish Shrenik Jain 2. Aayushi Jain 3. Meenakshi Kumari 4. Rawnak Kumar 5. Vivek K R

1. Executive Summary

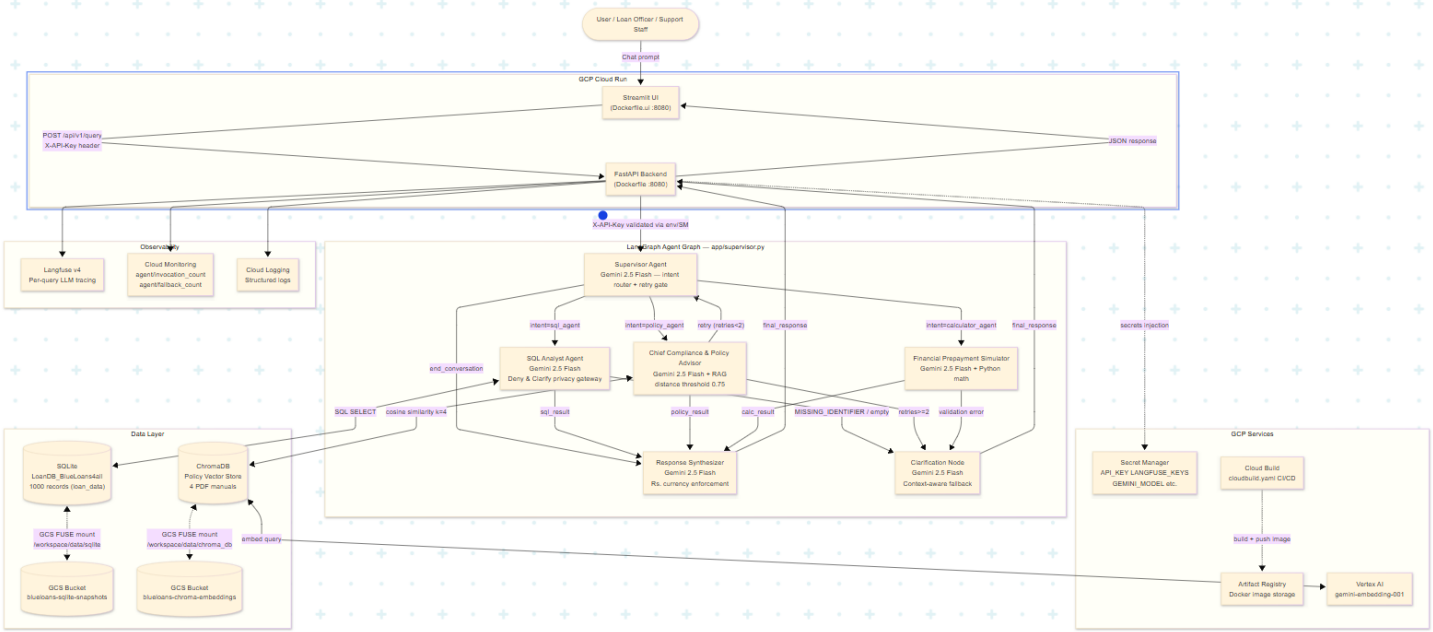
This document describes the architecture of the Loan Navigator Agent Suite — a production-grade multi-agent system built for BlueLoans4all, an Indian financial services company. In India's fast-paced fintech space, BlueLoans4all empowers micro-entrepreneurs with small-ticket loans. Their support teams handle high volumes of repetitive but critical queries: EMI status, prepayment scenarios, and top-up eligibility. This system automates those queries with an AI-powered Loan Navigator that is secure, cloud-native, and fully observable.

The system implements a Supervisor-Specialist Multi-Agent Pattern using LangGraph, with six collaborating nodes orchestrated through intent-driven conditional routing. The supervisor classifies user intent using Gemini 2.5 Flash and routes queries to three specialist agents: the SQL Analyst Agent, the Chief Compliance & Policy Advisor (RAG), and the Financial Prepayment Simulator. A Response Synthesizer compiles outputs into a single answer, and a Clarification Node handles unresolvable queries gracefully.

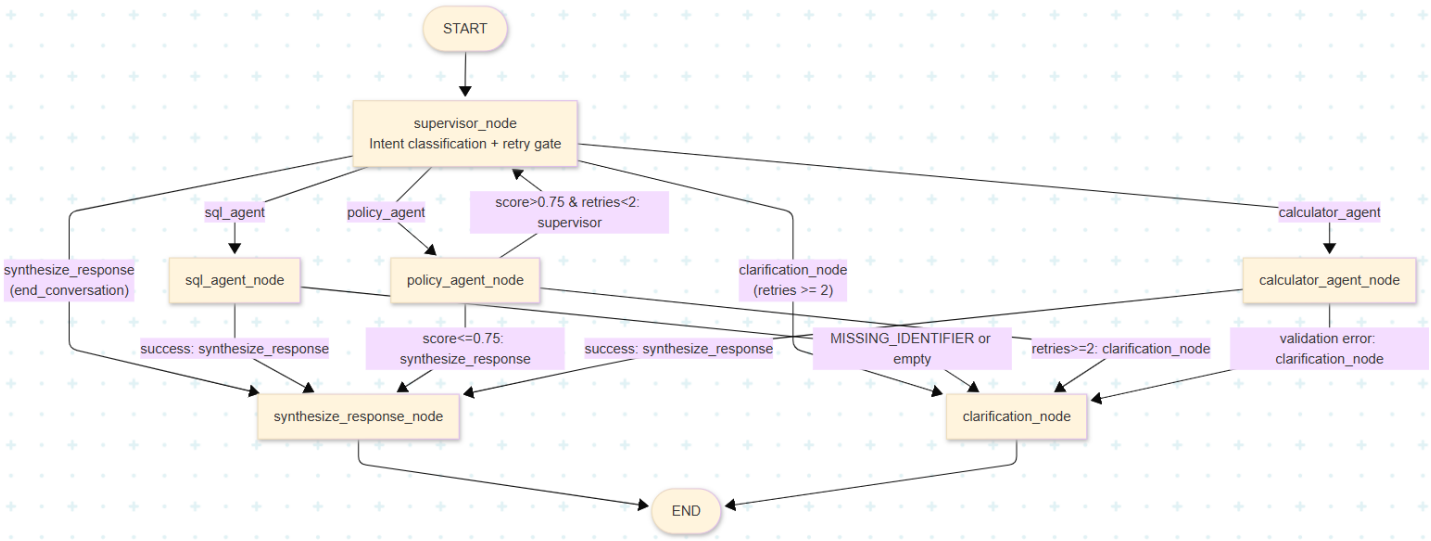
Key capabilities and architectural highlights:

* NLP-to-SQL over a 1,000-record SQLite loan database with a Deny & Clarify privacy gateway - queries without a specific loan ID or customer ID are refused and routed to clarification.
* Retrieval-Augmented Generation over four BlueLoans4all policy PDFs using ChromaDB and gemini-embedding-001, with cosine-distance threshold filtering and an automatic query – rewriting retry loop (max 2 retries).
* Deterministic prepayment simulation (Option A: reduce EMI; Option B: reduce tenure) with full amortisation schedules and rigorous Python-level input validation.
* Dual LLM backend: ChatGoogleGenerativeAI with vertexai = True for production (GCP), graceful local fallback using Google AI Studio API key.
* Containerised deployment on GCP Cloud Run with GCS FUSE volume mounts for data persistence, all secrets via GCP Secret Manager.
* End-to-end observability: Langfuse v4 (per-query LLM tracing) + GCP Cloud Monitoring (custom invocation and fallback metrics) + GCP Cloud Logging.
* Comprehensive test suite: 6 test files covering unit tests, integration tests, API endpoint tests, utility tests, and an interactive TEST_PLAYBOOK.md for UI verification.

2. Full System Architecture Diagram



3. LangGraph State Graph- Detailed Flow



```

## 3.1 Agent State Fields — LangGraph TypedDict

| Field | Type | Description |
|---|---|---|
| **messages** | Annotated[Sequence [BaseMessage], operator.add] | Accumulated conversation messages appended via operator.add. Human and System messages read by supervisor and all agent nodes. |
| **intent** | str | Routing intent from supervisor. Values: sql_agent \| policy_agent \| calculator_agent \| end_conversation. |
| **sql_result** | str | Result from SQL Agent. DB rows, no-results message, or 'No data found' (MISSING_IDENTIFIER path). Never returned raw to the user. |
| **policy_result** | str | Grounded policy answer from Policy Agent with cited PDF source filename and page number. |
| **calc_result** | str | Prepayment simulation result with Option A and Option B comparisons and amortisation summaries. May contain 'Validation Error:' prefix to trigger clarification. |
| **final_response** | str | User-facing synthesised response from synthesize_response_node or clarification_node. Returned in API QueryResponse. |
| **current_agent** | str | Routing signal written by each node, drives all conditional edges. Values: sql_agent \| policy_agent \| calculator_agent \| synthesize_response \| clarification_node. |
| **policy_retries** | int | Incremented when Policy Agent finds no chunks passing the 0.75 distance threshold. Gate at >= 2 forces clarification_node. |
| **clarification_needed** | bool | Set True when the system cannot resolve the query. Surfaced in API response as status='clarification_needed'. |
```

4. Query Flow Modes

The system supports three primary operational modes plus a Clarification fallback. All modes share the same LangGraph graph; routing is determined at runtime by the current agent field in AgentState.

4.1 SQL Mode — SQL Analyst Agent

The SQL Analyst Agent includes a Deny & Clarify privacy gateway. Queries without a specific loan ID or customer ID are refused before any SQL is generated.

Step	Component	Action / Payload
1	**User Query**	Natural language question about a specific loan, e.g. 'Show my loan details for loan ID LN2003' or 'What is my outstanding balance for customer 101?'
2	**Supervisor Node**	Gemini classifies intent=sql_agent via RouteQuery structured output. AgentState.current_agent set to sql_agent. Langfuse trace span started.
3	**Privacy Gateway**	SQL Agent prompt includes rule: if query does NOT contain a specific loan_id or customer_id, output exactly 'MISSING_IDENTIFIER' — no SQL generated. This prevents broad data dumps and protects customer PII.
4	**MISSING_IDENTIFIER Path**	If LLM outputs 'MISSING_IDENTIFIER': record_fallback_event('sql_agent','missing_identifier') called. current_agent=clarification_node, clarification_needed=True. User asked to provide their loan or customer ID.
5	**SQL Generation**	If identifier present: Gemini generates SELECT on loan_data. Loan IDs like 'LN2003' parsed to integer (2003) in WHERE clause. SQL cleaned of markdown delimiters.
6	**SQLite Execute**	LangChain SQLDatabase.run() executes cleaned query. Empty result ('[]') triggers record_fallback_event('sql_agent','no_db_results') and returns synthesize_response path.
7	**Synthesizer**	Gemini composes natural language answer. System prompt enforces Rs. symbol (rupee) for all monetary values. No raw SQL or column names in response.

4.2 Policy / RAG Mode — Chief Compliance & Policy Advisor

Step	Component	Action / Payload
1	**User Query**	Policy question, e.g. 'Can I prepay early on my BlueLoans4all loan?' or 'What are the documentation requirements for a top-up loan?'
2	**Policy Agent**	ChromaDB similarity_search_with_score called with k=4. Returns (Document, distance) pairs. Distance threshold 0.75: chunks with distance > 0.75 are discarded as low-confidence.

Step	Component	Action / Payload
3	**Threshold Check**	If no chunks pass: policy_retries incremented, SystemMessage added to state, current_agent=supervisor. Supervisor detects retry context, uses Gemini to rewrite the query with broader terms, re-routes to policy_agent.
4	**Context Assembly**	Passing chunks formatted as [Source: filename.pdf, Page: N] plus content. Up to 4 chunks assembled for LLM context window.
5	**Grounded Answer**	Gemini generates answer strictly grounded in provided context only ('Answer based ONLY on the provided policy context'). Source PDF filename and page number cited in response.
6	**Synthesizer**	Final natural language response synthesised from policy_result. Citations preserved in user-facing output.
7	**Retry / Clarification**	If policy_retries >= 2 with still no matching docs, current_agent=clarification_node. User asked to rephrase or told the topic is not covered by current policy manuals.

4.3 Calculator Mode — Financial Prepayment Simulator

Step	Component	Action / Payload
1	**User Query**	What-if question, e.g. 'Prepay 10,000 on 75,000 outstanding balance at 12% interest for 12 months' or 'What happens if I make a prepayment of Rs.15,000?'
2	**Parameter Extraction**	Gemini extracts CalculatorInput via JsonOutputParser (Pydantic schema): principal, interest_rate, tenure_months, prepayment_amount. Defaults applied for any missing fields.
3	**Input Validation**	Python range checks: all four values must be > 0; prepayment_amount must not exceed principal. Validation Error immediately sets current_agent=clarification_node — no simulation call made.
4	**Zero-Interest Support**	If interest_rate = 0.0, EMI = principal / tenure_months (standard division, avoids division by monthly_rate formula). Edge case tested in test_calculator.py.
5	**Option A: Reduce EMI**	New EMI computed on reduced principal for original tenure. Total repayment, interest saved, and abbreviated amortisation schedule (first 3 + last 3 months) compiled.
6	**Option B: Reduce Tenure**	Original EMI kept. New tenure computed iteratively until balance = 0. Months saved and total interest saved calculated.
7	**Synthesizer**	calc_result formatted string narrated by Gemini Synthesizer in plain language with Rs. symbol on all monetary amounts.

4.4 Clarification Mode — Fallback Intervention

The Clarification Node is triggered in four distinct scenarios:

* SQL Agent outputs MISSING_IDENTIFIER — user provided no loan ID or customer ID (privacy gateway).
* SQL Agent gets no database results for a valid query (no_db_results fallback).
* Policy Agent exhausts 2 retries without finding docs with distance <= 0.75.
* Calculator Agent receives invalid inputs (non-positive values, prepayment exceeds principal, or malformed LLM JSON).

The node uses a context-aware LLM prompt that identifies the failure reason from state fields and generates a polite, customer-friendly message. Internal agent names, retry counts, error traces, and SQL queries are never exposed. clarification_needed=True is set in the API response.

5. Codebase Overview

This section provides a file-by-file guide to the repository — what each file does, why it exists, and how the pieces connect. The codebase follows a clean separation of concerns: agents are pure routing/LLM logic, tools are pure computation, utils are pure infrastructure, and supervisor.py is the wiring layer.

5.1 Repository Structure

```
Loan-Navigator-Agent-Suite/
├── app/                          Core backend Python package
│   ├── agents/                   Specialist agent nodes
│   │   ├── calculator_agent.py   Parameter extraction + prepayment simulation
│   │   ├── policy_agent.py       ChromaDB RAG + citation grounding
│   │   └── sql_agent.py          NLP-to-SQL + privacy gateway
│   ├── tools/
│   │   └── calculator.py         Deterministic EMI + amortisation math
│   ├── utils/
│   │   ├── db.py                 SQLite connection (LangChain SQLDatabase)
│   │   ├── llm.py                ChatGoogleGenerativeAI dual-mode init
│   │   ├── monitoring.py         GCP Cloud Monitoring time-series writer
│   │   └── vector_store.py       ChromaDB + GoogleGenerativeAIEmbeddings
│   ├── main.py                   FastAPI app + API Key auth gate
│   ├── state.py                  AgentState TypedDict definition
│   ├── supervisor.py             LangGraph graph + all node logic
│   ├── requirements.txt          Python dependency matrix
│   └── .env.example              Environment variable template
├── data/
│   ├── LoanDB_BlueLoans4all.sqlite   1,000 loan records
│   ├── chroma_db/                    Persisted ChromaDB vector index
│   └── policy_docs/                  4 BlueLoans4all policy PDFs
├── test/                         Comprehensive test suite (6 files)
│   ├── test_calculator.py        EMI math + prepayment + agent node tests
│   ├── test_policy_loop.py       Retry loop + max-retries + query rewrite
│   ├── test_sql_agent.py         SQL Agent success + fallback + privacy gate
│   ├── test_main.py              FastAPI endpoint + API Key auth tests
│   ├── test_integration.py       End-to-end SQL flow integration test
│   └── test_utils.py             db.py + llm.py + vector_store.py + monitoring
├── documentation/                Prior version PDFs
```

```
├── architecture.md              Architecture overview markdown
├── TEST_PLAYBOOK.md             Interactive UI testing guide (4 categories)
├── check_langfuse.py            Langfuse connectivity diagnostic script
├── ingest.py                    Offline PDF -> ChromaDB ingestion script
├── streamlit_app.py             Streamlit chat UI
├── Dockerfile                   API service container (Cloud Run)
├── Dockerfile.ui                UI service container (Cloud Run)
├── cloudbuild.yaml              GCP Cloud Build CI/CD config
├── .gcloudignore                Files excluded from gcloud deploys
├── pytest.ini                   Test runner configuration
└──.coveragerc                  Code coverage configuration
```

5.2 Key File Explanations

app/supervisor.py — The Heart of the System

Contains everything that makes the LangGraph graph run: all six node functions (supervisor_node, sql_agent_node wrapper, policy_agent_node wrapper, calculator_agent_node wrapper, synthesize_response_node, clarification_node), the router() conditional edge function, the StateGraph wiring, and the run_supervisor() entry point that builds the Langfuse callback and invokes the compiled graph.

app/state.py — Shared Memory

Defines AgentState as a TypedDict with 10 fields. Every node reads from and writes to this single shared object. The messages field uses operator.add as a reducer so messages are accumulated rather than replaced across nodes.

app/agents/sql_agent.py — SQL Analyst Agent

Generates SQL against the loan_data table. The key addition in this version is the MISSING_IDENTIFIER privacy gateway: the system prompt instructs Gemini to output the literal string 'MISSING_IDENTIFIER' if no loan_id or customer_id is present in the query. The agent detects this token and short-circuits to the Clarification Node before touching the database.

app/agents/policy_agent.py — Chief Compliance & Policy Advisor

Runs ChromaDB similarity search with distance threshold 0.75. On low-confidence retrieval it increments policy_retries and returns to the supervisor for query rewriting. On success, it formats chunks as [Source: filename.pdf, Page: N] and grounds the LLM answer strictly in retrieved content. Citations are embedded in the response.

app/agents/calculator_agent.py — Financial Prepayment Simulator

Uses Gemini with JsonOutputParser to extract four financial parameters (principal, interest_rate, tenure_months, prepayment_amount) from natural language. Python-level validation runs before calling the simulation tool. The actual EMI and amortisation math happens in app/tools/calculator.py, not in the LLM.

app/tools/calculator.py — Deterministic Math Engine

Contains calculate_emi() and calculate_prepayment_impact(). These are pure Python functions with no LLM calls. calculate_prepayment_impact() generates two full amortisation schedules (Option A: reduce EMI; Option B: reduce tenure) with per-month payment breakdowns. Zero-interest rate is handled as a special case.

app/utils/llm.py — Dual-Mode LLM Client

Initialises ChatGoogleGenerativeAI (langchain-google-genai). If GCP_PROJECT_ID is set in the environment, it runs in Vertex AI mode (vertexai=True). If not set, it falls back to Google AI Studio using GOOGLE_API_KEY. This dual-mode design means the exact same code runs locally and in production without any changes.

app/utils/vector_store.py — Embedding & Vector Store

Initialises GoogleGenerativeAIEmbeddings with model gemini-embedding-001. When running locally without GCP_PROJECT_ID, it prepends 'models/' to the model name as required by the Google AI Studio API. Loads the persisted ChromaDB from data/chroma_db/.

app/utils/monitoring.py — GCP Metrics Writer

Writes custom time-series metrics to Google Cloud Monitoring via the monitoring_v3 client. Two public functions: record_agent_invocation(agent_name) and record_fallback_event(agent_name, reason). Both call _write_time_series() which creates a data point with value=1 on the relevant metric path. Gracefully skips metric writing if the GCP client cannot be initialised (local dev).

app/main.py — FastAPI Gateway

Exposes two endpoints: GET /health (liveness probe for Cloud Run) and POST /api/v1/query. The query endpoint validates the X-API-Key header against the API_KEY environment variable. If API_KEY is not set, it allows the request in local mock mode (useful for testing). On success, calls run_supervisor() and returns a QueryResponse with status='success' or status='clarification_needed'.

ingest.py — Offline Ingestion Script

Loads all PDFs from data/policy_docs/ using PyPDFDirectoryLoader. Splits into 1200-character chunks with 200-character overlap using RecursiveCharacterTextSplitter. Embeds each chunk with gemini-embedding-001 and persists to ChromaDB at data/chroma_db/. Run this script once before starting the backend, or whenever policy documents are updated.

check_langfuse.py — Observability Diagnostic

A standalone diagnostic script that reads your .env file, initialises the Langfuse CallbackHandler, sends a test message through Gemini, and reports whether the trace appears in the Langfuse cloud dashboard. Run this when first setting up the project or debugging observability issues.

TEST_PLAYBOOK.md — Interactive Testing Guide

A structured test guide with 4 categories (16 test cases): SQL Analyst Agent, Chief Compliance & Policy Advisor, Financial Prepayment Simulator, and Conversational Orchestration & Zero-Trust. Each test case provides a specific prompt, expected behavior, and which internal components should activate. Use this to manually verify the full system after deployment.

6. Component Table- Technology Justifications

Component	Technology	Justification
LLM — All Agents	Gemini 2.5 Flash ChatGoogleGenerativeAI temperature=0.2	Single model for all six nodes. vertexai=True in production (GCP_PROJECT_ID set), falls back to Google AI Studio locally (GOOGLE_API_KEY). Dual-mode design means the same codebase runs in both environments without modification.
Embeddings	gemini-embedding-001 GoogleGenerativeAIEmbeddings (768 dim)	Google's latest general-purpose embedding model. 'models/' prefix auto-prepended for local AI Studio API mode. Used in both ingest.py (offline) and Policy Agent (runtime). Native integration with LangChain.
Orchestration	LangGraph StateGraph (app/supervisor.py)	Stateful TypedDict graph with conditional edges, counter-based retry loops, and six distinct nodes. Retry cycle and MISSING_IDENTIFIER short-circuit expressed as conditional edges with state fields — clean, inspectable, and easy to extend.
Structured Data	SQLite (local dev) Cloud SQL / AlloyDB (prod) LangChain SQLDatabase	Zero-config SQLite for development with 1,000 loan records. LangChain SQLDatabase provides schema introspection and execution. GCS FUSE mount makes SQLite persistent on Cloud Run without code changes.
Vector Store	ChromaDB (local + prod) AlloyDB AI (prod option)	ChromaDB with cosine similarity and 0.75 distance threshold. GCS FUSE mount makes ChromaDB persistent on Cloud Run. AlloyDB AI optional for production for hybrid dense+BM25 search and native GCP IAM.
Privacy Gateway	LLM prompt constraint (MISSING_IDENTIFIER token)	SQL Agent prompt instructs Gemini to output 'MISSING_IDENTIFIER' if no loan_id or customer_id is present. Detected server-side and short-circuits to Clarification Node. Prevents broad data dumps and protects customer PII without any regex or rule engine.
Calculation Engine	Custom Python app/tools/calculator.py	Deterministic Python for all financial arithmetic. LLM role limited to parameter extraction and narration. Guarantees exact EMI accuracy, reproducible amortisation, and auditable results. Fully unit-tested including zero-interest edge case.
API Layer	FastAPI 0.100+ Uvicorn Python 3.11	/api/v1/query accepts QueryRequest (query + user_id), returns QueryResponse (response + status). API Key auth via X-API-Key header with Security dependency. /health for Cloud Run liveness. GCP Cloud Logging handler attached on startup.
Observability	Langfuse v4 GCP Cloud Monitoring GCP Cloud Logging	Langfuse: per-query LLM tracing with token counts, latency, user_id, and trace_name. GCP Monitoring: agent/invocation_count and agent/fallback_count custom time-series per agent. Cloud Logging: structured logs to Cloud Operations Suite via CloudLoggingHandler.
Secrets Management	GCP Secret Manager + .env (local dev)	All credentials (API_KEY, Langfuse keys, model names, paths) stored in Secret Manager for production. Injected as Cloud Run environment variables at deploy time. .env file for local dev,

Component	Technology	Justification
		excluded from git and gcloud uploads via .gitignore and .gcloudignore.
Data Persistence (Cloud Run)	GCS FUSE Volume Mounts (cloud-storage type)	Cloud Run volumes of type cloud-storage mount GCS buckets as file system paths. SQLite reads from /workspace/data/sqlite, ChromaDB reads from /workspace/data/chroma_db. No code changes needed versus local file paths.
CI/CD	GCP Cloud Build cloudbuild.yaml Artifact Registry	cloudbuild.yaml automates Docker image build and push to Artifact Registry for the UI service. API service built via direct gcloud builds submit. Images tagged with :latest for Cloud Run to pull.
User Interface	Streamlit 1.58 streamlit_app.py	Chat interface with st.chat_input, persistent session-state message history, and spinner during LLM calls. Calls FastAPI via HTTP with X-API-Key header. Suitable for internal loan officer tooling.

7. Non-Functional Requirements

7.1 Target Latency

Note: Local measurements exclude Vertex AI API round-trip time. Add 400-800 ms per LLM call in production. The Calculator Agent makes 2 LLM calls; Policy RAG + 1 retry makes 4 LLM calls total.

Query Type	P50 Target	P95 Target	Measured (local, no LLM)
SQL Only	900 ms	2,500 ms	~5 ms (SQLite read, no API call)
Policy / RAG	1,500 ms	4,000 ms	~8 ms (ChromaDB cosine search)
Calculator	1,200 ms	3,000 ms	~3 ms (pure Python, no API)
Policy RAG + Retry	3,000 ms	6,000 ms	~16 ms (two ChromaDB searches)
MISSING_IDENTIFIER / blocked	800 ms	2,000 ms	<1 ms (prompt token check, no DB call)

7.2 Throughput

* Target: 30 concurrent users, 120 queries/hour (internal loan officer tooling).
* Cloud Run scaling: min-instances=1 (warm), max-instances=10 per service. Independent scaling for API and UI containers.
* GCS FUSE adds ~10-50ms per file read. LLM latency (400-800ms) dominates; FUSE overhead is negligible.

7.3 Cost Estimate — 1,000 Queries / Day

Component	Usage	Est. Cost / Month
Gemini 2.5 Flash (all agents)	~3M tokens / day	~ $24
gemini-embedding-001	~200K tokens / day	~ $2
GCS Storage (SQLite + ChromaDB)	<10 GB persistent	~ $1
Cloud Run — API service	1 min instance + burst	~ $15
Cloud Run — UI service	1 min instance + burst	~ $10
Langfuse (cloud.langfuse.com)	~30K traces / month	~ $0 (free tier)
GCP Cloud Monitoring + Logging	Custom metrics + log ingestion	~ $5
TOTAL (production estimate)		**~ $57 / month**

8. Security Design

8.1 API Authentication

#	Control	Implementation
1	**X-API-Key Header**	All /api/v1/* endpoints require X-API-Key validated against API_KEY env var (from Secret Manager in production). Invalid or missing key returns HTTP 403 with message: 'Could not validate credentials: Invalid X-API-Key.'
2	**GCP Service Account**	Vertex AI calls authenticated via GOOGLE_APPLICATION_CREDENTIALS (ADC). Service account holds roles/aiplatform.user only.
3	**Secret Manager**	All credentials (API_KEY, Langfuse keys, model names, paths) in GCP Secret Manager. Injected as Cloud Run env vars at deploy time. Never baked into container images or committed to git.

8.2 SQL Injection & Privacy Protection

#	Control	Implementation
1	**Deny & Clarify Gateway**	SQL Agent prompt rule: if no specific loan_id or customer_id in query, output exactly 'MISSING_IDENTIFIER'. Detected server-side, routes to Clarification Node. Prevents broad data access (e.g., 'show all loans').
2	**Read-only SQLite Connection**	SQLDatabase.from_uri('sqlite:///path') opens in read-only mode. DML statements (INSERT, UPDATE, DELETE, DROP) fail at the database level even if generated by the LLM.
3	**Schema-bound Prompting**	SQL Agent system prompt explicitly lists only the loan_data table and its columns. sqlite_master and system table enumeration is outside the schema definition.
4	**No Raw SQL to User**	Generated SQL is logged to Langfuse traces but never included in the final_response returned to the UI or API caller.

8.3 Prompt Injection Defence

#	Technique	Implementation
1	**Role Isolation**	Each agent prompt is a server-side Python string constant with a tightly scoped role. No 'do anything' escape hatch exists. Prompts are never returned in API responses or logs.
2	**Structured Output (Pydantic)**	Supervisor uses LLM.with_structured_output(RouteQuery). Calculator uses JsonOutputParser(CalculatorInput). LLM output parsed into typed schema — free-text injection cannot bypass routing logic.

#	Technique	Implementation
3	**Policy Grounding Constraint**	Policy Agent system prompt: 'Answer based ONLY on the provided policy context below.' Prohibits drawing on training knowledge for regulatory content.
4	**No System Prompt Leakage**	System prompts are Python constants. Never included in API responses, Langfuse user-visible fields, or log messages surfaced to the Streamlit UI.

8.4 Output Safety

* Indian Rupee Enforcement: Synthesizer system prompt mandates Rs. symbol on all monetary values.
* Clarification-only Fallback: Clarification Node responses never reveal agent state, retry counts, error traces, or SQL queries.
* No Raw Database Rows: Synthesizer transforms raw sql_result strings into natural language - raw tabular data never reaches the user.
* Grounded Policy Answers: Policy Agent responses grounded strictly in retrieved chunks. Training-knowledge leakage on regulatory content is explicitly prohibited.

9. SLO Definitions- Production Recommendation

The following SLOs are recommended for a production deployment serving 30 concurrent loan officers at 120 queries/hour. All metrics measured at the FastAPI boundary from user request to final response delivery.

9.1 Availability SLO

Metric	Target	Measurement Method
Monthly Uptime	99.5%	Cloud Monitoring Uptime Check on /health every 5 minutes.
Error Budget	0.5% = 3.6 hours / month	Absorbs planned Cloud Run deployments and unplanned API outages.
Recovery Time	P95 < 5 minutes	Time from first PagerDuty alert to first successful /health probe.

9.2 Latency SLO

Query Type	P95 Target	P99 Target	PagerDuty Alert At
SQL queries	3,500 ms	5,000 ms	P95 > 5,000 ms
Policy / RAG queries	5,000 ms	7,000 ms	P95 > 7,000 ms
Calculator queries	4,000 ms	6,000 ms	P95 > 6,000 ms
Policy RAG + Retry	7,000 ms	10,000 ms	P95 > 10,000 ms
MISSING_IDENTIFIER / Clarification	2,000 ms	3,500 ms	P99 > 3,500 ms

9.3 Quality SLO

Metric	Target	Measurement
Policy retrieval precision	> 85%	20 random policy responses audited weekly. Citations verified against source PDFs.
SQL query correctness	> 90%	10% random sample reviewed weekly against expected DB results.

Metric	Target	Measurement
Calculator accuracy	100%	test_calculator.py run on every deployment. Zero regressions permitted.
Clarification rate	< 10%	Fraction of queries returning clarification_needed=True tracked via Langfuse. High rates indicate policy doc gaps or routing failures.
Privacy gateway precision	100% blocking	test_sql_agent.py::test_sql_agent_node_missing_identifier run on every deployment. MISSING_IDENTIFIER gate must block 100% of no-identifier queries.

9.4 Alert Thresholds

Condition	Severity	Response
P95 latency > 5s for any query type	SEV-2	PagerDuty alert. Check Vertex AI API latency and Cloud Run concurrency.
Error rate > 5% over any 5-minute window	SEV-1	PagerDuty + incident bridge. Check agent/fallback_count metric in GCP Monitoring.
policy_agent fallback rate > 20%	SEV-2	Slack alert. Review ChromaDB index freshness. Re-run ingest.py if policy PDFs changed.
sql_agent missing_identifier rate > 30%	SEV-3	Slack alert. Review supervisor prompt — may be misrouting queries to SQL that lack identifiers.
Langfuse trace gap > 10 minutes	SEV-3	Slack alert. Check CallbackHandler and LANGFUSE_PUBLIC_KEY in Secret Manager.

Appendix A- Policy Documents Ingested

Document	Coverage
BlueLoans4all_Loan_Policy_Manual.pdf	General loan origination criteria, eligibility, interest rate bands, standard terms and conditions.
BlueLoans4all_Risk_and_Underwriting_Policy.pdf	Credit risk assessment methodology, underwriting guidelines, LTV ratios, default management.
BlueLoans4all_Topup_and_Upgrade_Policy.pdf	Top-up loan eligibility criteria, repayment track record requirements, upgrade product rules.
BlueLoans4all_Digital_Infrastructure_Policy.pdf	Digital channel usage policies, data security requirements, acceptable-use rules.

Appendix B- Database Schema (loan_data table, 1,000 rows)

Column	Type	Description
loan_id	INTEGER	Unique loan identifier. Displayed as LN + id (e.g. LN2003). SQL Agent extracts integer part from user input.
customer_id	INTEGER	Customer identifier. Required for MISSING_IDENTIFIER gateway to pass.
loan_amount	REAL	Original sanctioned loan amount in Indian Rupees.
interest_rate	REAL	Annual interest rate as percentage (e.g. 11.5 for 11.5% p.a.).
start_date	DATE	Loan disbursement date.
tenure_months	INTEGER	Total loan tenure in months.
monthly_emi	REAL	Scheduled monthly EMI computed at origination.
amount_paid	REAL	Cumulative amount paid to date. Outstanding balance = loan_amount - amount_paid.
next_due_date	TEXT	Next EMI payment date. NULL for closed loans.
status	TEXT	Active | Closed.
topup_eligible	INTEGER	0 or 1. Boolean flag for top-up eligibility per policy criteria.

Column	Type	Description
prepayment_limit	REAL	Maximum allowed prepayment amount. 0.0 for loans with no prepayment facility.

Appendix C- Test Suite Summary

Test File	Scope	Key Tests
test_calculator.py	Unit + Agent	EMI formula, prepayment options A & B, loan closed path, zero-interest edge case, JSON extraction, validation errors, malformed JSON fallback.
test_policy_loop.py	Unit	Low-similarity fallback, max-retries safeguard (no LLM call), supervisor query rewrite for retry.
test_sql_agent.py	Unit	Successful SQL query, empty DB result fallback, exception handling, MISSING_IDENTIFIER privacy gateway.
test_main.py	API	/health check, no API_KEY set (mock mode), valid API key, invalid API key (403), clarification_needed status.
test_integration.py	Integration	End-to-end SQL flow: supervisor routing → SQL generation → DB execution → synthesis with Rs. symbol. Full LangGraph graph invoked.
test_utils.py	Unit	SQLDatabase file-not-found, success, exception; LLM production (vertexai=True) and local dev modes; vector store scenarios; monitoring write success/failure.